

Lab Assignment # 06

Course Title : **AI Assistant Coding**
Name of Student : **R Govardhan Sai Ganesh**
Enrollment No. : **2303A54044**
Batch No. : **48**

Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals

Task Description #1: Classes (Student Class)

Scenario: You are developing a simple student information management module.

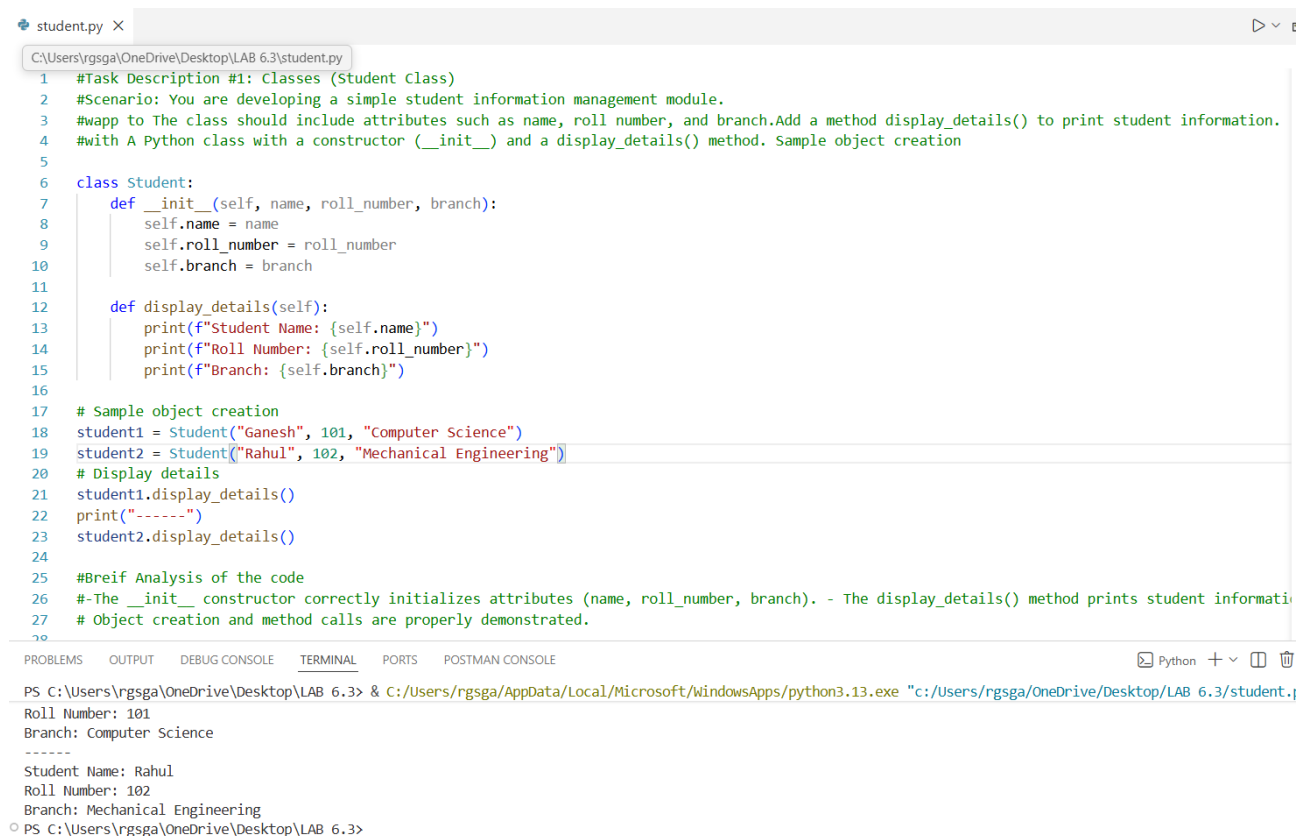
Task

- Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.
- The class should include attributes such as name, roll number, and branch.
- Add a method `display_details()` to print student information.
- Execute the code and verify the output.
- Analyze the code generated by the AI tool for correctness and clarity.

Expected Output #1

- A Python class with a constructor (`__init__`) and a `display_details()` method.
- Sample object creation and output displayed on the console.
- Brief analysis of AI-generated code.

Output Screenshot:



```
student.py X
C:\Users\rgsga\OneDrive\Desktop\LAB 6.3\student.py
1 #Task Description #1: Classes (Student Class)
2 #Scenario: You are developing a simple student information management module.
3 #wapp to The class should include attributes such as name, roll number, and branch.Add a method display_details() to print student information.
4 #with A Python class with a constructor (__init__) and a display_details() method. Sample object creation
5
6 class Student:
7     def __init__(self, name, roll_number, branch):
8         self.name = name
9         self.roll_number = roll_number
10        self.branch = branch
11
12    def display_details(self):
13        print(f"Student Name: {self.name}")
14        print(f"Roll Number: {self.roll_number}")
15        print(f"Branch: {self.branch}")
16
17 # Sample object creation
18 student1 = Student("Ganesh", 101, "Computer Science")
19 student2 = Student("Rahul", 102, "Mechanical Engineering")
20 # Display details
21 student1.display_details()
22 print("-----")
23 student2.display_details()
24
25 #Breif Analysis of the code
26 #-The __init__ constructor correctly initializes attributes (name, roll_number, branch). - The display_details() method prints student informati
27 # Object creation and method calls are properly demonstrated.
28
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\rgsga\OneDrive\Desktop\LAB 6.3> & C:/Users/rgsga/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/rgsga/OneDrive/Desktop/LAB 6.3/student.py"
Roll Number: 101
Branch: Computer Science
-----
Student Name: Rahul
Roll Number: 102
Branch: Mechanical Engineering
PS C:\Users\rgsga\OneDrive\Desktop\LAB 6.3>
```

Brief analysis of AI-generated code:

The `__init__` constructor correctly initializes attributes (name, roll_number, branch). The `display_details()` method prints student information in a clear format. Object creation and method calls are properly demonstrated.

Task Description #2: Loops (Multiples of a Number)

Scenario: You are writing a utility function to display multiples of a given number.

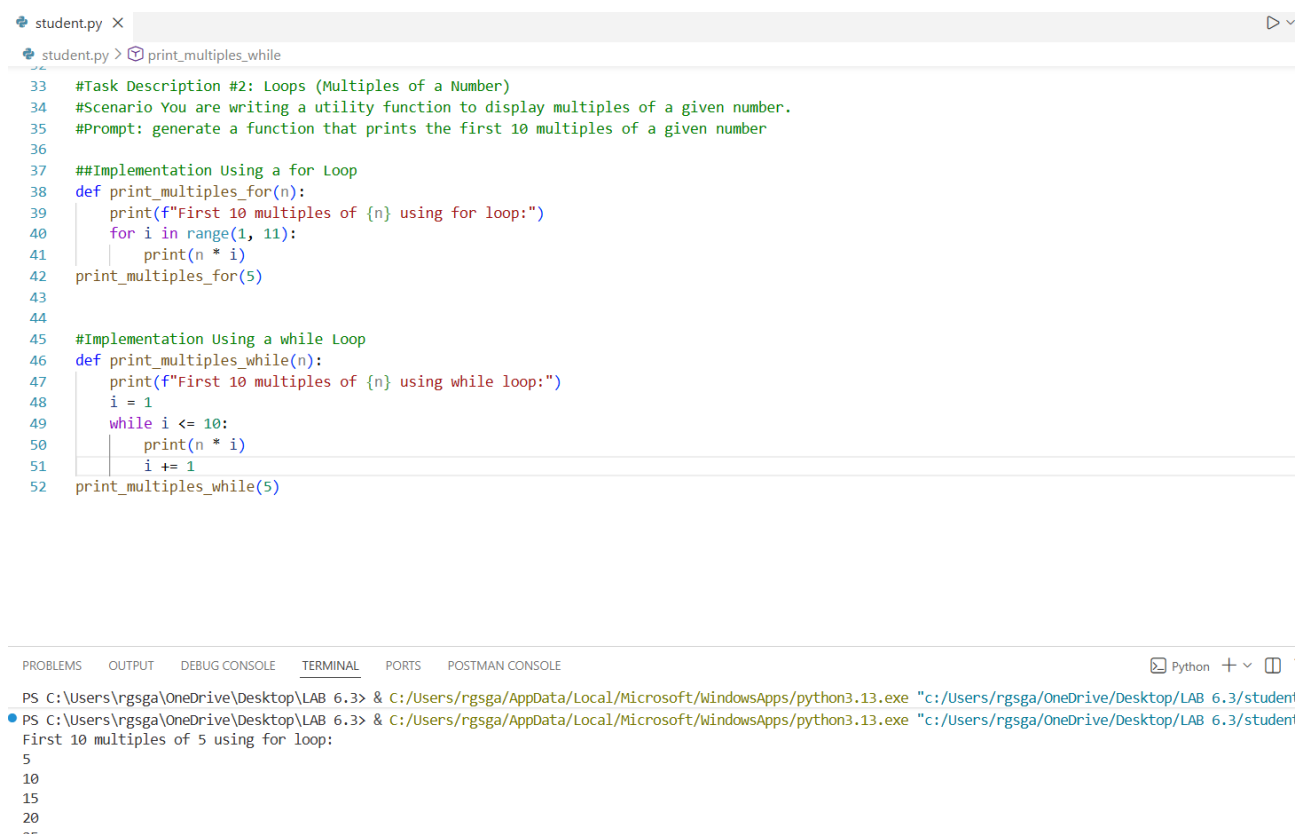
Task

- Prompt the AI tool to generate a function that prints the first 10 multiples of a given number using a loop.
- Analyze the generated loop logic.
- Ask the AI to generate the same functionality using another controlled looping structure (e.g., while instead of for).

Expected Output #2

- Correct loop-based Python implementation.
- Output showing the first 10 multiples of a number.
- Comparison and analysis of different looping approaches.

Output Screenshot:



```
student.py X
student.py > print_multiples_while

33 #Task Description #2: Loops (Multiples of a Number)
34 #Scenario You are writing a utility function to display multiples of a given number.
35 #Prompt: generate a function that prints the first 10 multiples of a given number
36
37 ##Implementation Using a for Loop
38 def print_multiples_for(n):
39     print(f"First 10 multiples of {n} using for loop:")
40     for i in range(1, 11):
41         print(n * i)
42 print_multiples_for(5)
43
44
45 #Implementation Using a while Loop
46 def print_multiples_while(n):
47     print(f"First 10 multiples of {n} using while loop:")
48     i = 1
49     while i <= 10:
50         print(n * i)
51         i += 1
52 print_multiples_while(5)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE Python + ▾

```
PS C:\Users\rsgsa\OneDrive\Desktop\LAB 6.3> & C:/Users/rsgsa/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/rsgsa/OneDrive/Desktop/LAB 6.3/student.py"
PS C:\Users\rsgsa\OneDrive\Desktop\LAB 6.3> & C:/Users/rsgsa/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/rsgsa/OneDrive/Desktop/LAB 6.3/student.py"
First 10 multiples of 5 using for loop:
5
10
15
20
~
```

Comparison and analysis of different looping approaches: The for loop is more elegant for fixed repetitions, while the while loop offers flexibility when the stopping condition isn't predetermined.

Task Description #3: Conditional Statements (Age Classification)

Scenario: You are building a basic classification system based on age.

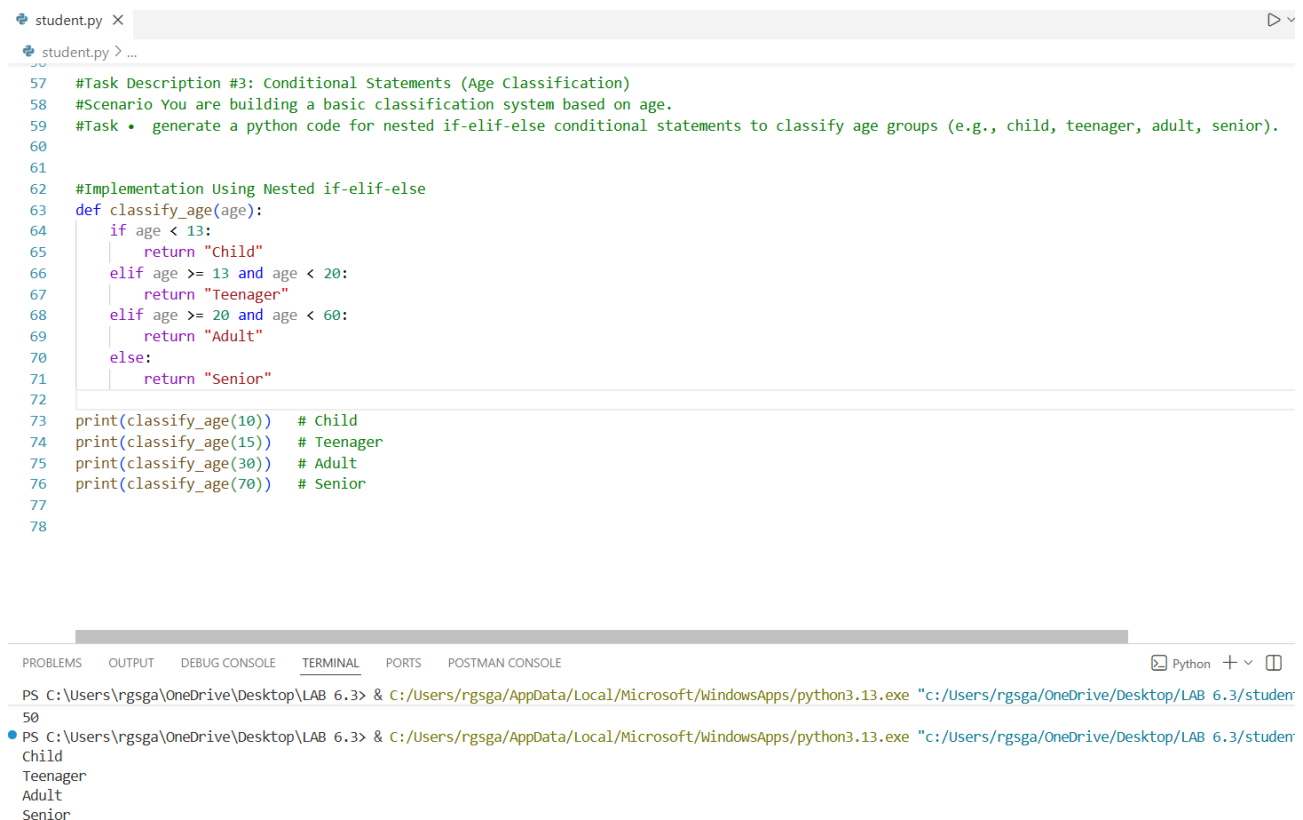
Task

- Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups (e.g., child, teenager, adult, senior).
- Analyze the generated conditions and logic.
- Ask the AI to generate the same classification using alternative conditional structures (e.g. simplified conditions or dictionary-based logic).

Expected Output #3

- A Python function that classifies age into appropriate groups.
- Clear and correct conditional logic.
- Explanation of how the conditions work.

Output Screenshot:



```
student.py X
student.py > ...
57 #Task Description #3: Conditional Statements (Age Classification)
58 #Scenario You are building a basic classification system based on age.
59 #Task • generate a python code for nested if-elif-else conditional statements to classify age groups (e.g., child, teenager, adult, senior).
60
61
62 #Implementation Using Nested if-elif-else
63 def classify_age(age):
64     if age < 13:
65         return "Child"
66     elif age >= 13 and age < 20:
67         return "Teenager"
68     elif age >= 20 and age < 60:
69         return "Adult"
70     else:
71         return "Senior"
72
73 print(classify_age(10)) # Child
74 print(classify_age(15)) # Teenager
75 print(classify_age(30)) # Adult
76 print(classify_age(70)) # Senior
77
78
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE Python + ▾

```
PS C:\Users\rgsga\OneDrive\Desktop\LAB 6.3> & C:/Users/rgsga/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/rgsga/OneDrive/Desktop/LAB 6.3/studen
50
● PS C:\Users\rgsga\OneDrive\Desktop\LAB 6.3> & C:/Users/rgsga/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/rgsga/OneDrive/Desktop/LAB 6.3/studen
Child
Teenager
Adult
Senior
```

Explanation:

1. If age < 13 → classify as ****Child****.
2. Else if age < 20 → classify as ****Teenager****, else if age < 60 → ****Adult****.
3. Any age ≥ 60 → classify as ****Senior****.

Task Description #4: For and While Loops (Sum of First n Numbers)

Scenario You need to calculate the sum of the first n natural numbers.

Task

- Use AI assistance to generate a `sum_to_n()` function using a for loop.
- Analyze the generated code.
- Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.

Expected Output #4

- Python function to compute the sum of first n numbers.
- Correct output for sample inputs.
- Explanation and comparison of different approaches.

Output Screenshot:



```
student.py > ...
96 #Task Description #4: For and While Loops (Sum of First n Numbers)
97 #Scenario You need to calculate the sum of the first n natural numbers.
98 #Task generate a python code to generate a sum_to_n() function using a for loop.
99
100 #Implementation using for loop
101 def sum_to_n_for(n):
102     total = 0
103     for i in range(1, n + 1):
104         total += i
105     return total
106
107 # Example usage
108 print(sum_to_n_for(5)) # Output: 15
109 print(sum_to_n_for(10)) # Output: 55
110
111
112 #implementation using while loop
113 def sum_to_n_while(n):
114     total = 0
115     i = 1
116     while i <= n:
117         total += i
118         i += 1
119     return total
120
121 # Example usage
122 print(sum_to_n_while(5)) # Output: 15
123 print(sum_to_n_while(10)) # Output: 55

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
Python + v []
PS C:\Users\rgsga\OneDrive\Desktop\LAB 6.3> & C:/Users/rgsga/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/rgsga/OneDrive/Desktop/LAB 6.3/student.f
Senior
PS C:\Users\rgsga\OneDrive\Desktop\LAB 6.3> & C:/Users/rgsga/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/rgsga/OneDrive/Desktop/LAB 6.3/student.f
15
55
15
55
```

Comparison of Approaches:

1. For Loop: Simple, explicit, easy to follow
2. While Loop: Flexible, can adapt to dynamic conditions

Explanation:

Loops calculate the sum step by step, while the formula gives the result instantly using math.

Task Description #5: Classes (Bank Account Class)

Scenario You are designing a basic banking application.

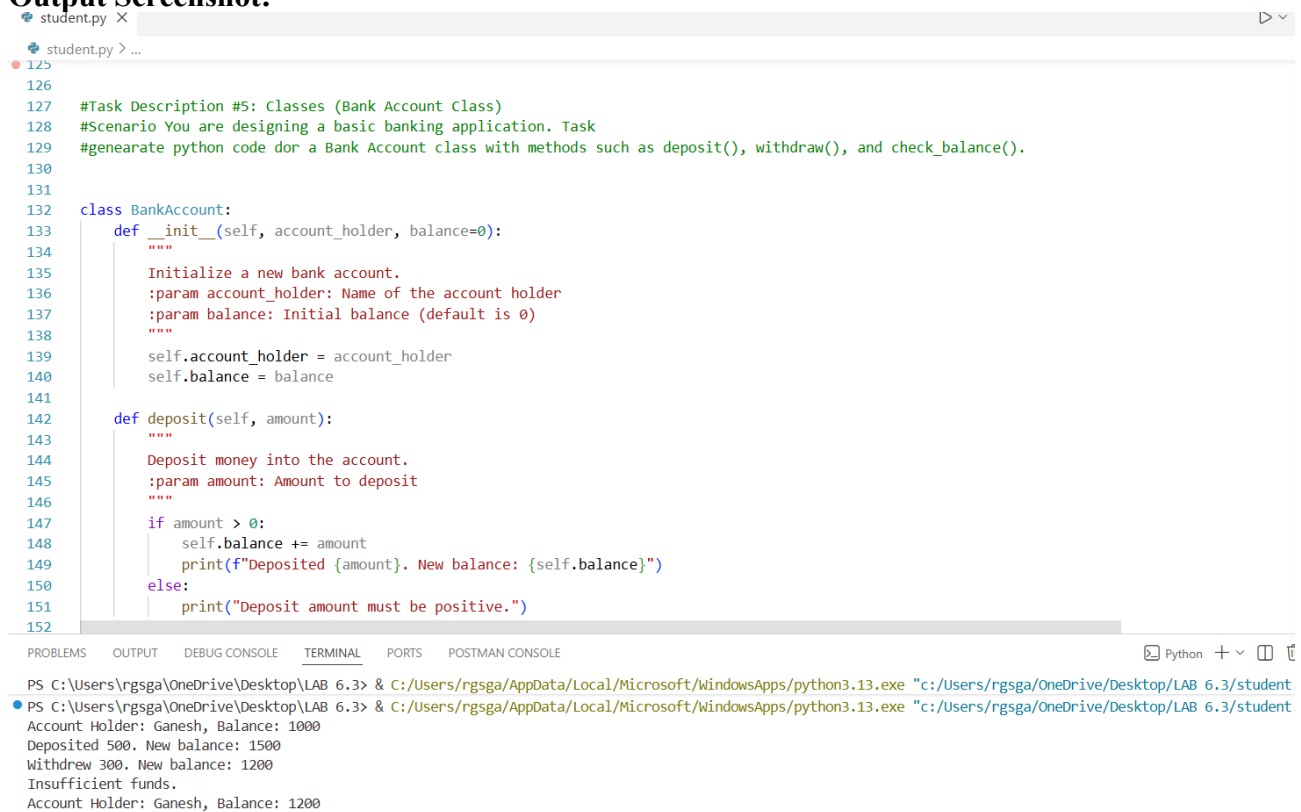
Task

- Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(), and check_balance().
- Analyze the AI-generated class structure and logic.
- Add meaningful comments and explain the working of the code.

Expected Output #5

- Complete Python Bank Account class.
- Demonstration of deposit and withdrawal operations with updated balance.
- Well-commented code with a clear explanation.

Output Screenshot:



```
student.py X
student.py > ...
student.py > ...
125
126
127 #Task Description #5: Classes (Bank Account Class)
128 #Scenario You are designing a basic banking application. Task
129 #generate python code for a Bank Account class with methods such as deposit(), withdraw(), and check_balance().
130
131
132 class BankAccount:
133     def __init__(self, account_holder, balance=0):
134         """
135         Initialize a new bank account.
136         :param account_holder: Name of the account holder
137         :param balance: Initial balance (default is 0)
138         """
139         self.account_holder = account_holder
140         self.balance = balance
141
142     def deposit(self, amount):
143         """
144         Deposit money into the account.
145         :param amount: Amount to deposit
146         """
147         if amount > 0:
148             self.balance += amount
149             print(f"Deposited {amount}. New balance: {self.balance}")
150         else:
151             print("Deposit amount must be positive.")
152
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
Python + v
PS C:\Users\rgsga\OneDrive\Desktop\LAB 6.3> & C:/Users/rgsga/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/rgsga/OneDrive/Desktop/LAB 6.3/student
Account Holder: Ganesh, Balance: 1000
Deposited 500. New balance: 1500
Withdrew 300. New balance: 1200
Insufficient funds.
Account Holder: Ganesh, Balance: 1200
```

Explanation of Code:

- Constructor (__init__) → Initializes account holder name and starting balance.
- deposit() → Adds money to the account if the amount is positive.
- withdraw() → Deducts money if the balance is sufficient; otherwise shows an error.
- check_balance() → Prints the current balance with account holder details.
- Demonstration → Shows deposit, withdrawal, and balance checking in action.